KLETIA

Autonomous USDC Commerce & Agentic DeFi on Arc

PROGRAMMABLE MONEY HACKATHON

ARC × ENCODE CLUB

FURKANAKDOGANFINANCE@GMAIL.COM

LIVE MVP	GITHUB	VIDEO DEMO	CHAIN
kletiaai.xyz	furkan3152/Kletia	youtu.be/5eiso1jkGiM	Arc Testnet (5042002)

01 EXECUTIVE SUMMARY

Kletia is an AI-powered decentralized finance and commerce hub deployed natively on **Arc Network**, Circle's purpose-built L1 where USDC is the native gas token. Users and autonomous AI agents interact through a natural language Intent Engine (GPT-4o) that maps commands into structured onchain execution — swapping, lending, staking, and mass payroll distribution.

The platform deploys **11 smart contracts** on Arc Testnet, featuring **7 active UI modules** and a fully integrated AI engine. Everything settles instantly in Native USDC with sub-second finality.

HACKATHON TRACK ALIGNMENT

- **DeFi Track:** 11 live smart contracts providing stablecoin-native AMM, lending, staking, yield vaults, mass payroll (BatchPay), and invoicing (MemoTransfer) using Native USDC.
- **Agentic Economy Track:** GPT-4o powered AI Intent Engine + onchain AgentRegistry. Autonomous agents hold wallets, parse natural language, and settle complex actions in USDC without a human in the loop.

ARC PRODUCTS USED

- **Arc Network L1** — Settlement layer (Malachite BFT)
- **Native USDC** — Gas token & settlement currency
- **EIP-7708 Transfer Logs** — Unified event indexing
- **Arc Testnet RPC** — rpc-testnet.arc.network

02 DEPLOYED CONTRACTS (ARC TESTNET)

All contracts are live and verified on Arc Testnet. The table below shows every deployed address.

CONTRACT	ADDRESS	STATUS
KletiaArcSwap	0x535EF89e3C3a74Cf1A76703972686cb7a2e34fe8	✓ UI + AI Engine
KletiaArcLending	0x2748a478Ec0f6D90FfdE89b27721f469126835F7	✓ UI + AI Engine
KletiaArcStaking	0xB85a7F6335D0544b4951e5f07Bcd326722b2BC07	✓ UI + AI Engine
KletiaArcVault	0xe2810DB53998f8A51bBf5Bf94c21208b174da174	✓ UI + AI Engine
KletiaArcBatchPay	0x09B6d2987EcAF021533A2727d2967696595Fa6dd	✓ UI Active
KletiaArcMemoTransfer	0x1633f12f31195B34feE6eDC250e1D543DAB72698	✓ UI + AI Engine
KletiaArcAgentRegistry	0xDEb07309c1689fEeCa44ac70939ce0297d511596	✓ AI Engine
KletiaArcOTC	0xf4c1F168491F22222145cff88414fE489BF8c39d	✓ Deployed
KletiaArcForwarder	0x7de7a249673f2235a91e484cfce49d00867b67ec	✓ Deployed
KletiaSmartRouter	0x8214b00F49Da60684ce4B2C0b16dDB8a29d777cf	✓ Deployed
KletiaToken (KLET)	0xAe77D247c26258397653a020995E957Bc88E039A	✓ Core Asset

03 PROTOCOL SUITE — CORE MODULES

SWAP — AMM + TWAP ORACLE

Constant-product (x·y=k) AMM with 0.3% fee. Issues ERC20 LP tokens. Uniswap V2-style cumulative price TWAP accumulators feed the Lending protocol's price oracle.

LENDING — AAVE-STYLE POOL

Ray/Wad math (10^{27}/10^{18}). Kinked rate model: 4% below 80% util, 300% above. LTV 70%, Liquidation Threshold 80%, 5% bonus. Health Factor monitoring.

STAKING — USDC YIELD

Stake Native USDC, earn APR-based rewards. Configurable cooldown for unstaking. Owner funds reward pool. `claimRewards()` distributes accrued USDC.

VAULT — COMPOUND APY

Deposit Native USDC into yield vault. Compound APY with real-time `pendingInterest()`. Emergency withdraw for instant exit. Owner manages vault funding.

BATCHPAY — MASS PAYROLL

Atomic batch USDC distribution with audit memos. Reverts entirely if any transfer fails. Bounded loop via `maxRecipientsPerBatch`. On-chain BatchRecord structs.

MEMOTRANSFER — INVOICING

USDC transfers with immutable on-chain memo strings (1–256 bytes). Dual-indexed history (sender + recipient). Self-transfer prevention.

AGENTREGISTRY — AI IDENTITY

ERC-8004 inspired on-chain registry for AI agents. Skill-based reverse indexing, reputation scoring (0–10,000 bps). Connected to AI intent engine.

OTC — P2P TRADING

Peer-to-peer limit order desk for ERC20 ↔ Native USDC. `createOrder()` locks maker funds; `fillOrder()` executes atomic swap.

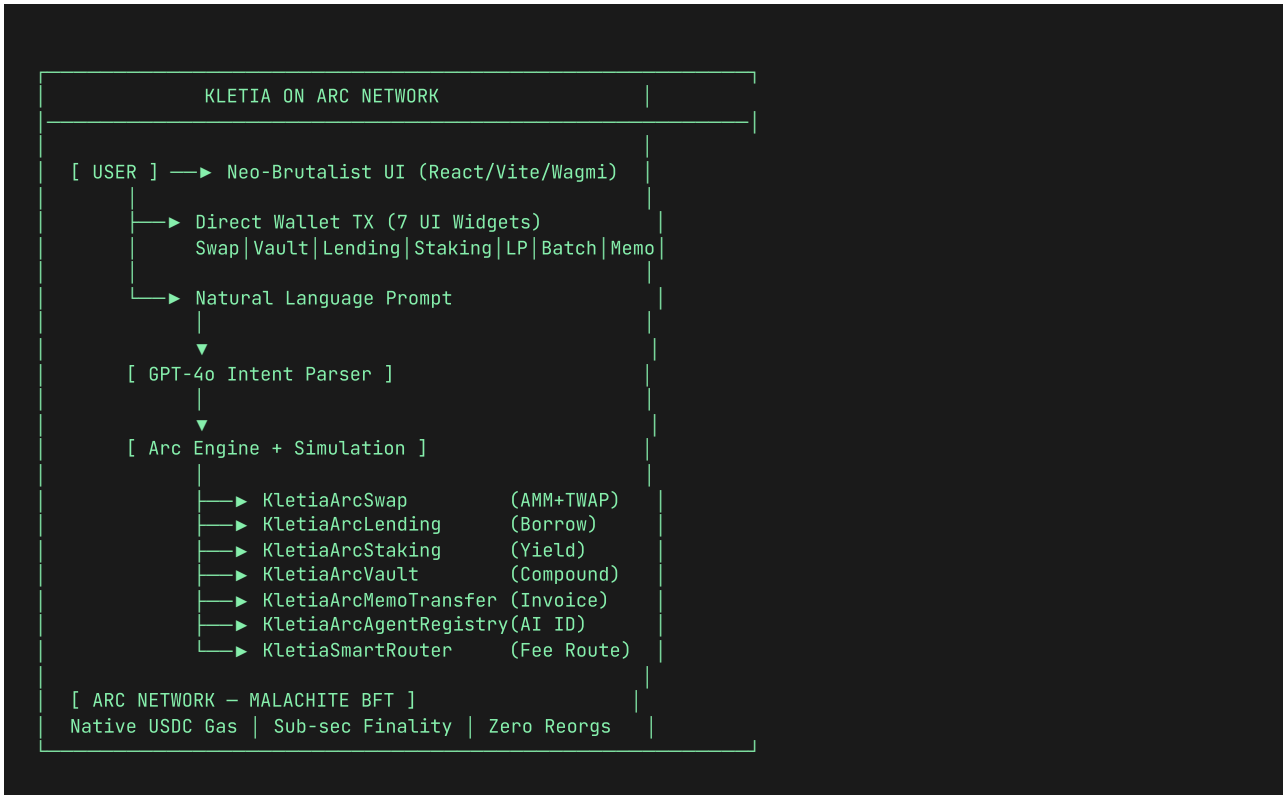
04 AGENTIC ECONOMY: AI INTENT ENGINE

Kletia's core differentiator for the Agentic Economy Track is its **GPT-4o powered intent engine**. Users and agents type natural language prompts ("swap 10 USDC for KLET", "stake 50 USDC") and the engine parses them into structured on-chain actions with pre-execution simulation.

```
USER PROMPT: "swap 5 USDC for KLET tokens"
──────────────────────────────────────────────────────────
STEP 1 | GPT-4o Parser → { action: "swap", amount: 5, direction: "usdcToToken" }
STEP 2 | arcXRaySimulate() → Pre-execution gas & revert check
STEP 3 | KletiaArcSwap.swapUSDCForToken({ value: 5 USDC })
STEP 4 | Event: Swapped(user, USDC, KLET, 5, tokenAmount)
STEP 5 | Real-time Socket.io → agentLog stream to UI
```

Supported Autonomous Actions: swap, stake, vault_deposit, vault_withdraw, memo_send, register_agent, add_liquidity, lending_deposit, borrow, repay.

05 SYSTEM ARCHITECTURE



06 WHY ARC NETWORK?

ARC FEATURE	KLETIA LEVERAGE
Native USDC Gas	Users pay gas in USDC directly. BatchPay and MemoTransfer costs are completely dollar-denominated, which is essential for mainstream adoption.
Sub-second Finality	Malachite BFT delivers deterministic finality with zero reorg risk. Critical for payroll settlement and OTC fills.
EIP-7708 Logs	Every native USDC movement emits standard ERC-20 Transfer events. Simplifies off-chain indexing for our APIs.
EWMA Fee Smoothing	Arc's Fee Manager smooths block-to-block gas price spikes. Our TWAP oracle updates on every swap need predictable execution costs.
Dual-Decimal Model	18-decimal native / 6-decimal ERC20 allows our Lending protocol's Ray/Wad math to operate at full 10^{27} precision natively.

07 PRODUCT FEEDBACK (DOCS.ARC.NETWORK)

✓ EVM DIFFERENCES PAGE — VERY HELPFUL

The EVM differences documentation clearly explains the dual-decimal USDC model, EIP-7708 behavior, and SELFDESTRUCT nuances. This page should be mandatory reading for all new builders — it prevented precision bugs in our Lending and Swap contracts.

△ DEVELOPER ONBOARDING COULD BE SMOOTHER

Setting up Hardhat to work with Arc Testnet required manual RPC configuration. A starter template or Hardhat plugin for Arc would significantly reduce setup friction. The Foundry integration documentation is solid.

△ DUAL-DECIMAL CONVERSION TOOLING

The 18-decimal native vs 6-decimal ERC20 interface is well-documented, but a Solidity helper library or SDK utility for safe conversions would prevent arithmetic errors that all builders will encounter when mixing native and ERC20 USDC calls.